

Transposed-Memory Free Implementation for Cost-Effective 2D-DCT Processor

Shih-Chang Hsia · Chin-Feng Tsai · Szu-Hong Wang · King-Chu Hung

Received: 24 October 2006 / Revised: 5 May 2008 / Accepted: 22 January 2009 / Published online: 14 March 2009
© 2009 Springer Science + Business Media, LLC. Manufactured in The United States

Abstract This paper presents a cost-effective 2D-DCT processor based on a fast row/column decomposition approach. With a particular schedule, the processor does not require the transposed memory for 2D-DCT computing. We re-arrange the cosine coefficients of the first and second 1D-DCT transformations to keep DC-coefficient error free. The new architecture uses state-machines to generate cosine coefficients rather than ROM table, to save the memory cells and the address generator. For 8×8 DCT realization, the circuit only needs 36 adders without multipliers, and the whole chip uses about 19 k transistors. The chip area is about 4 mm^2 using TSMC $0.35 \text{ }\mu\text{m}$ CMOS process. The circuit complexity is only $1/3 \sim 1/5$ of the conventional DCT chips.

Keywords DCT · Multiplier · Row/column decomposition · JPEG/MPEG · Video coding

1 Introduction

Discrete cosine transform (DCT) [1] had widely been used in image and video coding systems, such as JPEG[2], MPEG-2[3], H.263 [4]. However, design for a low-cost DCT processor is a challenge task for its high computation.

In order to realize DCT efficiently, many fast algorithms [5–7] are presented to reduce the computational complexity. Basically, there are two approaches for DCT implementation. One is the row/column decomposition, and the other is the direct 2D transform. The row/column approach consists of two 1D-DCTs and one transposed memory to realize a 2D-DCT chip. As a fast 1-D algorithm is employed [8], the processing complexity can be further halved. On the other hand, some algorithms explore about the fast direct 2D DCT/IDCT computations, but their complexity is still higher than that of 1-D processing scheme [9–10]. Hence a dedicated DCT processor with the row/column approach is more popular for real-time VLSI implementation [11–14]. There have been various designs proposed for row/column-based DCT. The distributed arithmetic design [11], used ROM table and accumulator rather than multiplier to improve the processing speed. The multiplier-based [12] employed the common 1-DCT unit for 2D-DCT computation to reduce the hardware cost. The parallel structure [13] for high-speed design can achieve N coefficients per cycle, but the circuit complexity is too high. The bit-by-bit arithmetic approach [14] can reduce the circuit complexity, but the word length would constrain the processing speed. The pipeline architecture [15] is designed to achieve high efficiency for data transposition.

Currently, DCT/IDCT function is always implemented with the single chip since they have similar computations. However, in real-time video coding systems [2–4], the encoder needs the individual DCT and IDCT cores, but the decoder only requires IDCT. For better performance, the implementation of DCT and IDCT could be separated. Since many DCT coefficients would be quantized to zeros, one can use coefficient-by-coefficient approach to realize an IDCT processor to promote the processing speed greatly [9]. However, this approach is not available in DCT design.

This work was supported by National Science Council, Republic of China, under Grant NSC92-2213-E-327-010.

S.-C. Hsia (✉) · C.-F. Tsai · S.-H. Wang · K.-C. Hung
Department of Electronics Engineering and Department of
Computer and Communication Engineering, National Kaohsiung
First University of Science and Technology,
Kaohsiung 824 Taiwan, Republic of China
e-mail: hsia@ccms.nkfust.edu.tw

Table 1 Cosine coefficients for DCT implementation.

	Function	Near value	With 4-bit presentation
a	$\cos(\pi/4)$	0.70707	0.6875=0.1011
b	$\cos(\pi/16)$	0.98078	0.9375=0.1111
c	$\cos(\pi/8)$	0.92388	0.8750=0.1110
d	$\cos(3\pi/16)$	0.83147	0.8125=0.1101
e	$\cos(5\pi/16)$	0.55557	0.5=0.1000
f	$\cos(3\pi/8)$	0.38268	0.375=0.0110
g	$\cos(7\pi/16)$	0.1951	0.1875=0.0011

Fortunately, the computational accuracy for DCT is not important as IDCT in practical coding systems. Because IDCT results are used for decoding image, the differences of encoding and decoding will be accumulated to the frame memory and the degradation of image will be more and more serious as the coding process goes on. However, DCT coefficients would be further quantized to reduce the bit-rate. The truncated error is greatly larger than the transformed error. We do not require high-accuracy DCT transformation in applications of image coding. Based on this reason, for cost-effective design, a word length for DCT implementation can be efficiently shortened to reduce the chip complexity.

The row/column transformation always requires a block memory for data transposition. The memory occupies a large silicon area in practical VLSI design. To realize the memory, we can directly utilize registers from the cell-library. The circuit will become large since one register needs about 20 transistors in a typical cell. Instead, the embedded memory can be used, however, the chip implementations become more complex. To overcome this drawback, we design a 2D-DCT processor without using transposed memory. For cost-effective design, the word length can be further reduced while keeping the transformed accuracy with DC-coefficient correction approach. This paper is organized as follows. A cost-effective DCT algorithm is presented in Section 2. VLSI architecture and chip implementation is described in Section 3. Conclusions are addressed in Section 4.

2 Proposed Cost-Efficient DCT Algorithm

In video coding standards [3–4], the 2D-DCT formula is commonly defined as

$$Y_{uv} = C_u C_v \sum_{j=0}^{N-1} \sum_{k=0}^{N-1} d_{jk} \cos\left(\frac{(2j+1)u\pi}{2N}\right) \cos\left(\frac{(2k+1)v\pi}{2N}\right) \quad (1)$$

where d_{jk} is a spatial pixel, the constants, C_u and C_v , are defined as

$$C_{u(v)} = \begin{cases} 1/\sqrt{2} & , \quad uv = 0 \\ 1 & , \quad u, v \neq 0 \end{cases} \quad (2)$$

The row/column decomposition for 2D-DCT computation can be expressed as

$$\begin{aligned} Y_{uv} &= C_u \sum_{j=0}^{N-1} \left(C_v \sum_{k=0}^{N-1} d_{jk} \cos\left(\frac{(2k+1)v\pi}{2N}\right) \right) \cos\left(\frac{(2j+1)u\pi}{2N}\right) \\ &= C_u \sum_{j=0}^{N-1} (F_v) \cos\left(\frac{(2j+1)u\pi}{2N}\right) \\ &= 1D - DCT_{column}(F_v), \end{aligned} \quad (3)$$

and

$$\begin{aligned} F_v &= C_v \sum_{k=0}^{N-1} d_k \cos\left(\frac{(2k+1)v\pi}{2N}\right) \\ &= 1D - DCT_{row}(d_k), \quad \text{for } v = 0..7 \end{aligned} \quad (4)$$

Table 2 Cosine function modification in the 2nd DCT.

	Function	Near value	With 4-bit presentation
$\hat{a}$	$\sqrt{2}/2\cos(\pi/4)$	0.5	0.5000=0.1000(no error)
$\hat{b}$	$\sqrt{2}/2\cos(\pi/16)$	0.69348	0.6875=0.1011
$\hat{c}$	$\sqrt{2}/2\cos(\pi/8)$	0.65325	0.625=0.1010
$\hat{d}$	$\sqrt{2}/2\cos(3\pi/16)$	0.58791	0.5625=0.1001
$\hat{e}$	$\sqrt{2}/2\cos(5\pi/16)$	0.39283	0.3750=0.0110
$\hat{f}$	$\sqrt{2}/2\cos(3\pi/8)$	0.27058	0.250=0.0100
$\hat{g}$	$\sqrt{2}/2\cos(7\pi/16)$	0.13795	0.1250=0.0010

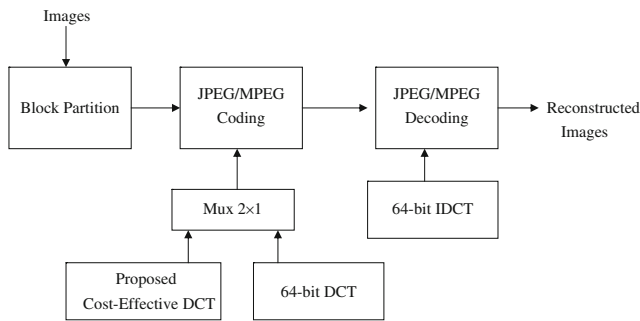


Figure 1 The block diagram of the environment in performing the accuracy estimation under JPEG/MPEG applications.

is 1D-DCT for the row pixel processing. $1D-DCT_{column}$ and $1D-DCT_{row}$ is one dimension DCT computation with column and row direction, respectively. As N rows are processed, the 1-D DCT coefficients are stored to memory as

$$F_{uv} = M_{uv} = \begin{bmatrix} M_{00} & M_{01} \dots & M_{06} & M_{07} \\ M_{10} & M_{11} \dots & M_{16} & M_{17} \\ \dots & \dots & \dots & \dots \\ M_{60} & M_{61} \dots & M_{66} & M_{67} \\ M_{70} & M_{71} \dots & M_{76} & M_{77} \end{bmatrix}_{8 \times 8} \quad \text{for } u, v = 0..7 \quad (5)$$

Then the second 1D-DCT is used to process the column

direction from the memory. The complete 2D-DCT from (3) can be expressed by

$$Y_{uv} = 1D - DCT_{column}(M_{uv}) = 1D - DCT_{row}(M_{uv})^T \quad (6)$$

where $(M_{uv})^T$ is the transposition of M_{uv} . We can transpose the result of 1D-DCT from memory, and then can employ two row-based 1D-DCTs to realize 2D-DCT. Since the cosine function has the symmetrical property, two input data can be pre-merged before cosine computing [8]. When $N=8$, a fast 1-D DCT algorithm can be presented by

$$\begin{bmatrix} F_{u0} \\ F_{u2} \\ F_{u4} \\ F_{u6} \end{bmatrix} = \begin{bmatrix} a & a & a & a \\ c & f & -f & -c \\ a & -a & -a & a \\ f & -c & c & -f \end{bmatrix} \begin{bmatrix} d_{j0} + d_{j7} \\ d_{j1} + d_{j6} \\ d_{j2} + d_{j5} \\ d_{j3} + d_{j4} \end{bmatrix} \quad (7)$$

and

$$\begin{bmatrix} F_{u1} \\ F_{u3} \\ F_{u5} \\ F_{u7} \end{bmatrix} = \begin{bmatrix} b & d & e & g \\ d & -g & -b & -e \\ e & -b & g & d \\ g & -e & d & -b \end{bmatrix} \begin{bmatrix} d_{j0} - d_{j7} \\ d_{j1} - d_{j6} \\ d_{j2} - d_{j5} \\ d_{j3} - d_{j4} \end{bmatrix} \quad (8)$$

where a, b, c, d, e, f and g are cosine coefficients given by

$$[a, b, c, d, e, f, g] = [\cos(\pi/4), \cos(\pi/16), \cos(\pi/8), \cos(3\pi/16), \cos(5\pi/16), \cos(3\pi/8), \cos(7\pi/16)] \quad (9)$$

Based on the fast 1D-DCT algorithm, a particular computing schedule is proposed to implement 2D-DCT without using transposed memory. As an 8×8 spatial block is given as

$$d_{jk} = \begin{bmatrix} d_{00} & d_{01} \dots & d_{06} & d_{07} \\ d_{10} & d_{11} \dots & d_{16} & d_{17} \\ \dots & \dots & \dots & \dots \\ d_{60} & d_{61} \dots & d_{66} & d_{67} \\ d_{70} & d_{71} \dots & d_{76} & d_{77} \end{bmatrix}, \quad \text{for } j, k = 0..7, \quad (10)$$

from (5) and (7–8), we first compute M_{00} by

$$M_{00} = a(d_{00} + d_{07}) + a(d_{01} + d_{06}) + a(d_{02} + d_{05}) + a(d_{03} + d_{04}). \quad (11)$$

Then one can compute M_{70} from

$$M_{70} = a(d_{70} + d_{77}) + a(d_{71} + d_{76}) + a(d_{72} + d_{75}) + a(d_{73} + d_{74}). \quad (12)$$

Table 3 Accuracy estimation with various cosine word-lengths for JPEG system.

	64-bit Length	3-bit Length		4-bit Length		6-bit Length		8-bit Length		12-bit Length	
		Non DC Corr.	DC Corr.	Non DC Corr.	DC Corr.	Non DC Corr.	DC Corr.	Non DC Corr.	DC Corr.	Non DC Corr.	DC Corr.
Flower	28.148	20.731	25.361	22.322	27.608	24.671	27.899	27.150	27.954	27.988	28.085
Football	31.130	21.291	28.081	23.451	30.765	25.620	30.804	30.409	31.056	30.879	31.079
Lady	35.314	22.983	32.050	25.675	34.805	29.701	34.963	34.621	35.092	34.812	35.183
Salesman	34.232	22.390	31.596	25.421	33.885	28.882	33.984	33.631	34.012	33.970	34.121
Average PSNR	32.206	21.849	29.272	24.217	31.766	27.219	31.913	31.452	32.029	31.912	32.117

“Corr.” is “Correction” abbreviation

Table 4 Accuracy estimation with averaged PSNR for MPEG-2 system.

Bit rate	64-bit length	3-bit length		4-bit length		6-bit length		8-bit length		12-bit length	
		Non DC Corr.	DC Corr.	Non DC Corr.	DC Corr.	Non DC Corr.	DC Corr.	Non DC Corr.	DC Corr.	Non DC Corr.	DC Corr.
600 k	27.326	20.349	25.512	21.390	27.027	24.391	27.120	26.421	27.203	27.156	27.243
800 k	28.840	21.931	26.126	22.451	28.371	25.239	28.452	27.893	28.541	28.479	28.679
1 M	30.560	22.679	28.034	24.397	30.126	27.892	30.301	29.882	30.405	30.012	30.474
1.2 M	32.019	24.034	29.892	26.561	31.789	28.778	31.852	30.033	31.903	31.431	31.945

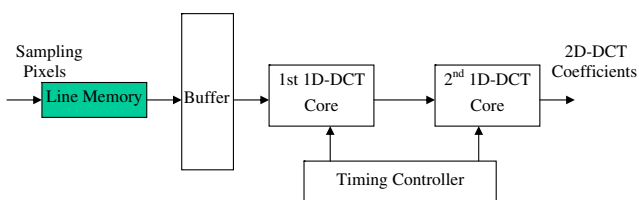
As M_{00} and M_{70} have been computed, the second 1D-DCT can start to transform. The temporal output coefficients can be computed by

$$\begin{bmatrix} \tilde{Y}_{00} \\ \tilde{Y}_{02} \\ \tilde{Y}_{04} \\ \tilde{Y}_{06} \end{bmatrix} = \begin{bmatrix} a \\ c \\ a \\ f \end{bmatrix} [M_{00} + M_{70}] \quad \text{and} \quad \begin{bmatrix} \tilde{Y}_{01} \\ \tilde{Y}_{03} \\ \tilde{Y}_{05} \\ \tilde{Y}_{07} \end{bmatrix} = \begin{bmatrix} b \\ d \\ e \\ g \end{bmatrix} [M_{00} - M_{70}] \quad (13)$$

Next, one can compute M_{10} and M_{60} by the same (11) and (12) procedures. Now the cosine weighting of $(M_{10} + M_{60})$ and $(M_{10} - M_{60})$ can be accumulated into the temporal output coefficients $\tilde{Y}_{00} \sim \tilde{Y}_{07}$ by the same (11)–(13) procedures. Repeatedly, $(M_{20}$ and $M_{50})$ and $(M_{30}$ and $M_{40})$ are computed. Their results are successively accumulated into the output registers. Finally, we can obtain the 2D coefficient of the first column. With the same way, one can compute the second column coefficients by computing M_{01} to M_{71} . Repeating this procedure, an 8×8 block can be transformed to 2D-DCT coefficients without using transposed memory.

2.1 Finite Word Length Reduction for Cost Effective Design

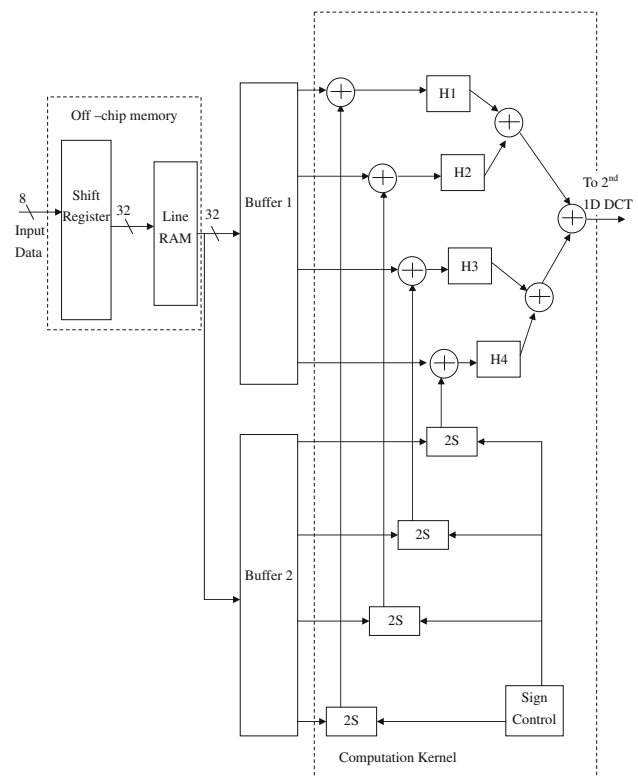
Currently, no criteria or standards are specified for the DCT computational accuracy. For cost-effective design, the word length can be shortened to reduce the chip complexity. However, we need to guarantee the image quality accept-

**Figure 2** Proposed DCT architecture without transpose memory.

able from practical coding systems while using a short word-length in DCT implementation. If the cosine coefficients are truncated to i -bit, each value can be expressed as

$$\text{Cos} = \sum_{k=1}^i S_k 2^{-k}, \quad (14)$$

where S_k is a binary code that is either zero or one. The cosine value is easily realized for $k=1$ to i , by right-shifting $1 \sim i$ bits. The near cosine function for (9) can be shown in Table 1. If a pixel uses 8 bits, the pre-processing will generate the 9-bit data and then multiplying by the cosine

**Figure 3** The architecture for the first 1-D DCT computation.

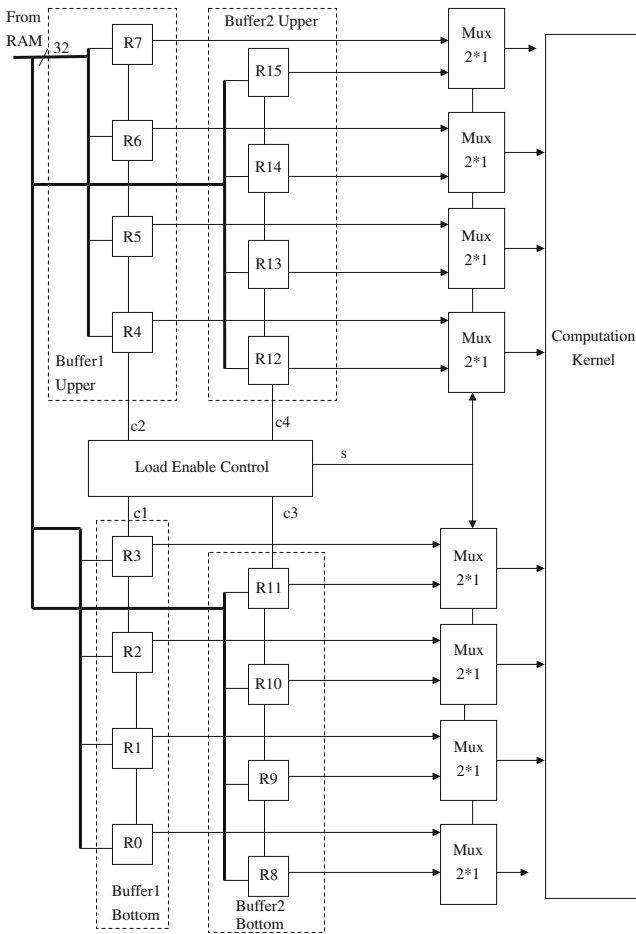


Figure 4 The buffer access control.

coefficient with (7) and (8). When $i=4$, the result can be expressed with four sections as

$$\begin{aligned} Mult_{coarse} &= d_{jk}[8..0] \times Cos \\ &= d_{jk}[8..1]S_1 + d_{jk}[8..2]S_2 + d_{jk}[8..3]S_3 \\ &\quad + d_{jk}[8..4]S_4. \end{aligned} \quad (15)$$

where Cos is a cosine coefficient. The $d_{jk}[8..0]$ is a 9-bit spatial pixel, where $d_{jk}[8]$ is MSB bit. In (15), $Mult$ is the coarse result of multiplication that is an integer since the floating number is shifted out, so there are serious truncation errors. To reduce the error, the first MSB of the floating number of each section is added to (15) as

$$\begin{aligned} Mult_{fine} &= Mult_{coarse} + d_{jk}[0] + d_{jk}[1] + d_{jk}[2] \\ &\quad + d_{jk}[3], \end{aligned} \quad (16)$$

where $d_{jk}[n-1]$ is the first MSB of the floating number to compensate the $d_{jk}[8..n]$ shifting from (15), $n=1$ to 4. For example, $d_{jk}[3]$ is a maximum floating bit in the last

section. To achieve a fine multiplication result, the $d_{jk}[3]$ is added to $Mult_{coarse}$ in order to reduce the truncated error from $d_{jk}[8..4]$. With this concept, DCT function is implemented using C-programming according to Eqs. (7)–(16) to analyze the computational accuracy. If the cosine coefficient only uses 4 bits, there is a serious deviation in the DC coefficient because the error will be successively accumulated from the deviation of positive “a” factor. Fortunately, transformed errors are small in AC coefficients since the error will be cancelled each other from the accumulation of positive and negative cosine weighting. To overcome the transformed error for DC coefficient, the fast algorithm from (7) is modified for the 1st DCT as

$$\begin{bmatrix} M_{j0} \\ M_{j2} \\ M_{j4} \\ M_{j6} \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ c & f & -f & -c \\ a & -a & -a & a \\ f & -c & c & -f \end{bmatrix} \begin{bmatrix} d_{j0} + d_{j7} \\ d_{j1} + d_{j6} \\ d_{j2} + d_{j5} \\ d_{j3} + d_{j4} \end{bmatrix}. \quad (17)$$

And the cosine weight for first row of 2nd DCT is changed to

$$\begin{bmatrix} Y_{00} \\ Y_{02} \\ Y_{04} \\ Y_{06} \end{bmatrix} = \begin{bmatrix} \hat{a} & \hat{a} & \hat{a} & \hat{a} \\ \hat{c} & \hat{f} & -\hat{f} & -\hat{c} \\ \hat{a} & -\hat{a} & -\hat{a} & \hat{a} \\ \hat{f} & -\hat{c} & \hat{c} & -\hat{f} \end{bmatrix} \begin{bmatrix} M_{00} + M_{70} \\ M_{10} + M_{60} \\ M_{20} + M_{50} \\ M_{30} + M_{40} \end{bmatrix} \text{ and} \quad (18)$$

$$\begin{bmatrix} Y_{01} \\ Y_{03} \\ Y_{05} \\ Y_{07} \end{bmatrix} = \begin{bmatrix} \hat{b} & \hat{d} & \hat{e} & \hat{g} \\ \hat{d} & -\hat{g} & -\hat{b} & -\hat{e} \\ \hat{e} & -\hat{b} & \hat{g} & \hat{d} \\ \hat{g} & -\hat{e} & \hat{d} & -\hat{b} \end{bmatrix} \begin{bmatrix} M_{00} - M_{70} \\ M_{10} - M_{60} \\ M_{20} - M_{50} \\ M_{30} - M_{40} \end{bmatrix}.$$

where the modified cosine coefficients $[\hat{a}, \hat{b}, \dots, \hat{g}] = \sqrt{2}/2[a, b, \dots, g]$. Table 2 shows the modified cosine values. The cosine coefficients of 2nd ~ 8th rows ($Y_{2v} \sim Y_{7v}$) in the 2nd DCT are not changed, where we still use (7) and (8). According to the new computation schedule in (17), the DC component in the 1st DCT can be obtained from

$$M_{j0} = \sum_{k=0}^7 d_{jk}, \quad \text{where } j = 0 \text{ to } 7. \quad (19)$$

Then these results are processed using the 2nd DCT in (18). Since $\hat{a} = 0.5 (= \sqrt{2}/2 \times \sqrt{2}/2)$, we can attain the DC coefficient from

$$DC = Y_{00} = \frac{1}{2} \sum_{j=0}^7 M_{j0} \quad (20)$$

From (19) and (20), the cosine coefficient is 1 and 0.5 for computing the DC coefficient of the first and second 1D-DCT units, respectively. Now the transformation for DC coefficient becomes error free for cosine values with 1 and 0.5.

Table 5 The timing schedule of input and output in the proposed processor.

clk	Read from RAM	S	1st 1D DCT computing	Output latch	2nd 1D DCT computing	Accu0~3	Accu4~7	Output enable
1	d ₀₀ ~d ₀₃ (R0~R3)	x						
2	d ₀₄ ~d ₀₇ (R4~R7)	x						
3	d ₇₀ ~d ₇₃ (R8~R11)	0	M[00] (from d ₀₀ ~d ₀₇)					
4	d ₇₄ ~d ₇₇ (R12~R15)	0						
5	d ₁₀ ~d ₁₃ (R0~R3)	1	M[70] (from d ₇₀ ~d ₇₇)	M[00]				
6	d ₁₄ ~d ₁₇ (R4~R7)	1						
7	d ₆₀ ~d ₆₃ (R8~R11)	0	M[10] (from d ₁₀ ~d ₁₇)		M[00]+M[70]			
8	d ₆₄ ~d ₆₇ (R12~R15)	0						
9	d ₂₀ ~d ₂₃ (R0~R3)	1	M[60] (from d ₆₀ ~d ₆₇)	M[10]	M[00]-M[70]	****		
10	d ₂₄ ~d ₂₇ (R4~R7)	1						
11	d ₅₀ ~d ₅₃ (R8~R11)	0	M[20] (from d ₂₀ ~d ₂₇)		M[10]+M[60]	****		
12	d ₅₄ ~d ₅₇ (R12~R15)	0						
13	d ₃₀ ~d ₃₃ (R0~R3)	1	M[50] (from d ₅₀ ~d ₅₇)	M[20]	M[10]-M[60]	****		
14	d ₃₄ ~d ₃₇ (R4~R7)	1						
15	d ₄₀ ~d ₄₃ (R8~R11)	0	M[30] (from d ₃₀ ~d ₃₇)		M[20]+M[50]	****		
16	d ₄₄ ~d ₄₇ (R12~R15)	0						
17	d ₀₀ ~d ₀₃ (R0~R3)	1	M[40] (from d ₄₀ ~d ₄₇)	M[30]	M[20]-M[50]	****		
18	d ₀₄ ~d ₀₇ (R4~R7)	1						
19	d ₇₀ ~d ₇₃ (R8~R11)	0	M[01] (from d ₀₀ ~d ₀₇)		M[30]+M[40]	****		
20	d ₇₄ ~d ₇₇ (R12~R15)	0						
21	d ₁₀ ~d ₁₃ (R0~R3)	1	M[71] (from d ₀₀ ~d ₀₇)	M[01]	M[30]-M[40]	****		Y[00]~Y[10]
22	d ₁₄ ~d ₁₇ (R4~R7)	1						Y[20]~Y[30]
23	d ₆₀ ~d ₆₃ (R8~R11)	0	M[11] (from d ₀₀ ~d ₀₇)		M[01]+M[71]	****		Y[40]~Y[50]
24	d ₆₄ ~d ₆₇ (R12~R15)	0						Y[60]~Y[70]
25	d ₂₀ ~d ₂₃ (R0~R3)	1	M[61] (from d ₀₀ ~d ₀₇)	M[11]	M[01]-M[71]	****		

2.2 Accuracy Estimation for Real Applications

For cost-effective design, we attempt to find the minimum word-length for image coding applications. The word length of cosine coefficients uses 3~12 bits to test the DCT accuracy. For comparisons, the true DCT computation with 64-bit length is as a reference. The DCT coefficients are further truncated to 12-bit for a real chip design [8–12]. Figure 1 shows the testing block diagram estimating the accuracy with JPEG/MPEG systems. When DCT processor is used for JPEG compression, 12-bit DCT coefficients are quantized according to JPEG quantization table. We employ four images to compare the reconstructed quality from 64-bit inverse DCT transformation. The PSNR results are shown in Table 3. As the cosine coefficient only used 3-bit, PSNRs degrade about 2 dB compared to the 64-bit

transformation. However, when the number of bit increases to 4, the coding quality becomes acceptable since the averaged PSNR only degrades about 0.5 dB after DC-coefficient correction. Besides, we hardly recognize the reconstructed image from the 64-bit transformation or the proposed method with 4-bit length. We also evaluate the proposed cost-effective DCT for MPEG-2 video coding applications. The testing condition is the same as [16] that uses the adaptive video coding control scheme. The averaged PSNRs are listed in Table 4 in various coding bit-rates. We also satisfy the coding accuracy with 4-bit word length after DC-coefficient correction. The 4-bit length is the lowest bound to keep the acceptable image quality in experiments. The accuracy can be improved if the word length increases, but we need to pay higher hardware cost too.

Table 6 The cosine function for H1~H4 modular in the 1st DCT.

	Cosine coefficient sequence*								State machine sequence
H1	1	b	c	d	a	e	f	g	16→15→14→13→11→8→6→3
H2	1	d	f	-g	-a	-b	-c	-e	16→13→6→3→11→5→14→8
H3	1	e	-f	-b	-a	-g	c	d	16→8→6→15→11→3→14→13
H4	1	g	-c	-e	a	-d	-f	-b	16→3→14→8→11→13→6→15

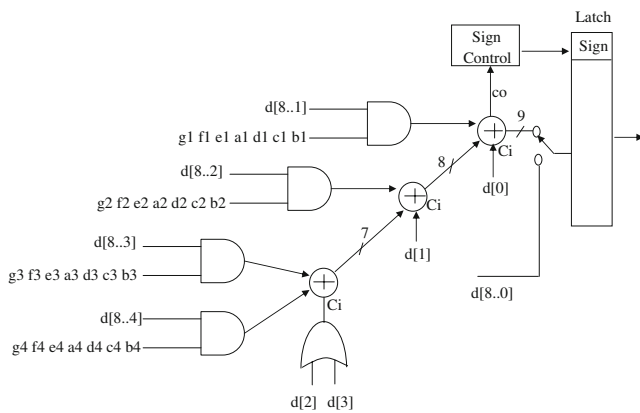


Figure 5 Implementation for H1 in the 1st DCT.

3 VLSI Architecture and Chip Implementation

Based on the cost-effective algorithm, we can design a low-cost DCT chip for practical video coding systems. Figure 2 illustrates the top layer of proposed DCT architecture. Since video is sequentially sampled with line-by-line, video data must be stored to the memory for block-based processing. The size of line memory needs at least $N \times (F_s/F_h)$ pixels, where F_s and F_h is the sampling frequency and the horizontal scanning frequency respectively, and the block size is $N \times N$. For block data access, first we read the line-memory data to the on-chip buffer. The 1st 1D-DCT reads the buffer and computes the 1-D coefficient, then the 1-D results are sent to the 2nd DCT to compute the final 2-D coefficient without using the transposed memory.

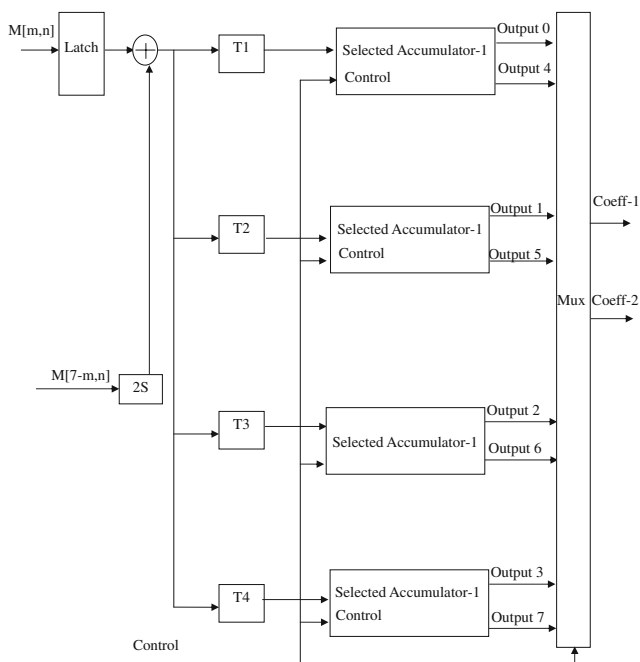


Figure 6 The computation kernel of the 2nd 1D-DCT.

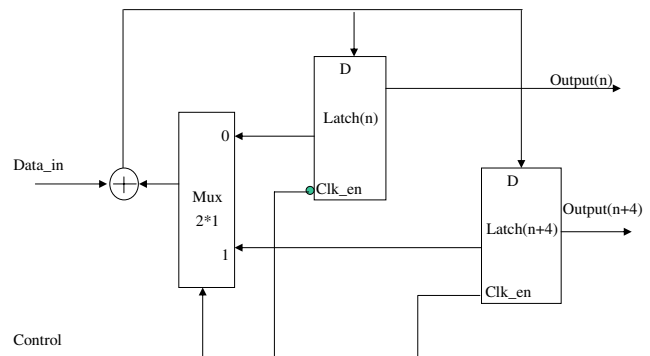


Figure 7 The selected accumulator module.

Figure 3 illustrates the architecture for the first 1D-DCT. The buffer employs the shift register to load the sampling data. The pixel inputs to the shift register in each clock. When the RAM width uses 32-bit and the image pixel is 8-bit resolution, we require to the temporal buffer to store the pixel before writing to the memory. After four clocks, the buffer can gather 32-bit data, and then the four pixels can be written to RAM at once. In order to read RAM data for real-time operation, on-chip buffer1 and buffer2 are employed. The computational kernel is realized with (7) and (8), where H1~H4 are the cosine weighting function. The buffer access control is shown in Fig. 4. Since each register has 8-bit, we can load 32-bit data to four registers in parallel. The buffer1 is partitioned to the upper (R4~R7) and bottom (R0~R3) parts, and the buffer2 is also split to the upper (R12~R15) and bottom (R8~R11) parts. At a time, we select a pair of registers for loading data from RAM. Only one signal of C1~C4 is high during one cycle, to control the registers, (R0~R3), (R4~R7), (R8~R11) and (R12~R15), respectively, Table 5 shows the timing schedule of the DCT processor. In the first and the second clocks, the pixels $d_{00} \sim d_{03}$ and $d_{04} \sim d_{07}$ are written to (R0~R3) and (R4~R7) respectively. At the third clock, $d_{70} \sim d_{73}$ are loaded to R8~R11. Simultaneously, we control the multiplex 2×1 by “s” signal for selecting R0~R7 input to the DCT computational kernel. Now we can compute M_{00} in the first 1D-DCT. At the 4th clock, $d_{74} \sim d_{77}$ are loaded to R12~R15. With this scheduling, one can obtain the M_{00} result in the 5th clock. By the same procedures, M_{70} is finished at the 7th clock. Now M_{00} and M_{70} can be sent to the second 1D-DCT unit for computing the 2D-DCT coefficients. At the 22th clock, we can finish two 2D-DCT coefficients $Y[00] \sim Y[10]$ for the first row. The other coefficients can be obtained in the next clocks. Hence the latency time of the proposed 2D-DCT is 22 clocks. In this timing schedule, the reusability of intermediate data is twice, and so the I/O speed is double the internal computing speed. Fortunately, the delay of I/O is less than that of computing circuit since the critical path occurs in the

Table 7 The cosine function for T1~T4 modular in the 2nd DCT.

	Cosine coefficient sequence*	State machine sequence
T1	$\hat{a} \ \hat{b} \ \hat{a} \ \hat{d} \ \hat{a} \ \hat{e} \ \hat{a} \ \hat{g}$	8→11→8→9→8→6→8→2 (1st row)
T2	$\hat{a} \ \hat{b} \ \hat{a} \ \hat{d} \ \hat{a} \ \hat{e} \ \hat{a} \ \hat{g}$ $\hat{c} \ \hat{d} \ \hat{f} \ -\hat{g} \ -\hat{f} \ -\hat{b} \ -\hat{c} \ -\hat{e}$	11→15→11→13→11→8→11→3 (2nd to 8th row)
T3	$\hat{c} \ \hat{d} \ \hat{f} \ -\hat{g} \ -\hat{f} \ -\hat{b} \ -\hat{c} \ -\hat{e}$ $\hat{a} \ \hat{e} \ -\hat{a} \ -\hat{b} \ -\hat{a} \ -\hat{g} \ \hat{a} \ \hat{d}$	10→9→4→2→4→11→10→6 (1st row)
T4	$\hat{a} \ \hat{e} \ -\hat{a} \ -\hat{b} \ -\hat{a} \ -\hat{g} \ \hat{a} \ \hat{d}$ $\hat{f} \ \hat{g} \ -\hat{c} \ -\hat{e} \ \hat{c} \ \hat{d} \ -\hat{f} \ -\hat{b}$	14→13→6→3→6→15→14→8 (2nd to 8th row)
		8→6→8→11→8→2→8→9 (1st row)
		11→8→11→15→11→3→11→13 (2nd to 8th row)
		4→2→10→6→10→9→4→11 (1st row)
		6→3→14→8→14→13→6→15 (2nd to 8th row)

computational kernel. Since DCT computation used the block-based, we require the temporal memory to store the line data. For data re-order, the particular schedule is used to access the memory to meet our scheduling with Table 5. Thus we employ the temporal registers R0~R15 to store the pixels for data re-ordering. The memory access and addressing design can refer to our previous paper [17].

3.1 1D-DCT Processing Unit

In order to save the transposed memory, the first and second 1D-DCTs adopt different architectures. For the 1st 1D-DCT unit, a sign-control modular determines the addition or subtraction of two data with two's complement (2S). The computational modular H1~H4 are the cosine weighting multiplications in Fig. 3. According to (7) and (8), one can use the state machine to generate the cosine coefficients, $\hat{a}$ ~ $\hat{g}$, for each module according to Table 6, where 4-bit cosine value is used (referred to Table 5). For cost-effective design, the DCT processor is design with multiplierless architecture. From (15)–(17), we can design the computational modular H1~H4. For example, Fig. 5 shows the architecture for H1 generation. For computing M_{00} , the input data is directly passed to the accumulator by a selector since the cosine coefficient is 1 from (17). Next,

the cosine coefficient is changed to “b” from Table 6, where “b” is combined with bits b1~b4 and b4 is the least significant bit. With (15), we can control whether the shifted data is added. In fact, only one 5-bit adder, one 6-bit adder and one 7-bit adder are realized for this module. The H2~H4 modular structure is similar to H1, but we need to change the input control function S_k by referring cosine coefficients. Since each control signal has eight states, we can employ a 3-bit state-machine to generate the fixed cosine coefficients rather than ROM table, where the sequence of state machine is shown in the right of Table 6. The final results are latched with registers to avoid the delay propagating to the second 1D-DCT.

Figure 6 shows the architecture of the 2nd DCT unit. One can attain the results of $M[m,n]$ and $M[7-m,n]$ from the 1st DCT unit in each cycle. The modular T1~T4 are for computing the cosine weighting in (18). The results of T1~T4 are sent to the selected accumulators in parallel. The modular of the selected accumulator is shown in Fig. 7. We employ two registers to accumulate the results of n^{th} and $(n+4)^{\text{th}}$ for each coefficients computing. The computation of a set of $M[m,n]$ and $M[7-m,n]$ can be finished during two cycles. In the first cycle, the multiplex 2x1 selects the n^{th} latch. The result from processing $(M[m,n] + M[7-m,n])$ is accumulated to the n^{th} register as the control signal is low. In the next cycle, the $(M[m,n] - M[7-m,n])$ is computed in T1~T4. Since the control signal becomes high, the results are accumulated to the $(n+4)^{\text{th}}$ register. The structure of computation modulars T1~T4 is similar, but the cosine

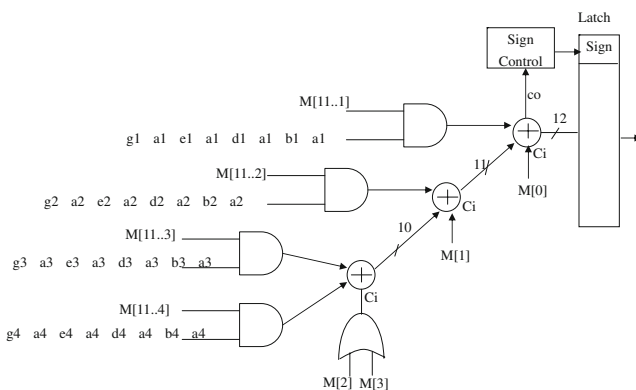
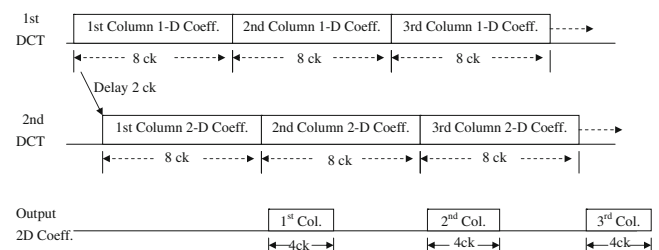
**Figure 8** Implementation for T1 in the 2nd DCT.**Figure 9** The computational timing of proposed DCT.

Table 8 Chip complexity evaluation.

	Number of Transistors		Occupation percentage
	1st 1D-DCT	2nd 1D-DCT	
Adder ^a	3,528	4,344	43%
Register ^a	3,608	4,972	47%
Multiplex	512	312	4%
Sign change	556	460	5%
Timing control	156	0	1%
Total	18,448		

^a 1 bit adder and 1 bit register with 24 and 22 Transistors respectively

coefficients are different, as shown in Table 7. The state machine sequence for T1~T4 is split into two parts, where one being for the 1st row and the other being for the 2nd to 8th rows. Figure 8 illustrates the T1 modular that can be implemented with three adders, where the architecture is similar to H1 but without using selector. Since the output clock is double to the computational clock, we can output 2D coefficients with the two parallel ports during two internal clocks. The detail timing can refer the Fig. 9. Therefore, this processor is capable to transform N coefficients during N cycles to meet real-time requirement.

3.2 Chip Realization

Based on the proposed architecture, we use Verilog HDL to realize the DCT processor [18]. The circuit is synthesized with TSMC 0.35 μm CMOS standard cells. Then CADENCE tools are used for the chip layout with auto placement and routing procedures. Each sub-circuit occupies the area listed for 1st and 2nd DCT unit in Table 8. The first and the second DCT core employs 19 and 17 adders respectively. In total, the proposed 8×8 DCT IP only employs 36 adders. The number of transistors for adders is 7,872, which occupied about 43% chip area. The input required 16×9 registers for re-order temporal processing in Fig. 4, where each input signal is 9-bit (included 1 signed bit). The output used 8×16 registers for data accumulation

in Fig. (6) and (7), where each accumulator employed 16-bit to reduce the truncated error. Moreover, we also require registers to generate the state machine for cosine function and some registers to store the computational result of each stage to improve the speed. The total register cells occupied about 47% chip area. The adder and register share the most hardware sources in the DCT chip. The other circuits, such as multiplex, sign change and control, only use about 10% area. The arithmetic elements efficiency can achieve 100%, where the hardware source is active in every cycle. Table 9 shows the chip features of proposed DCT. The total circuit of the cost-efficient DCT is only about 19 k transistors, and the core size is about 4 mm^2 silicon area. The chip includes six power pads, 38 input pads that have 9×4 input data pins and two clock pins, and two output ports with $2 \times 12 = 24$ pins that each coefficient resolution is 12 bits. The throughput of this chip achieves 60 MHz that are enough to apply on current video coding systems [2–4].

For comparisons, many techniques about DCT implementation have been reviewed in [1]. In general, the computational unit and the memory cell are two main circuits in DCT processors. The proposed 2D-DCT saves the transpose memory and shortens the word length to reduce the hardware complexity. Of course, the other DCT processors also can reduce its word length, but their transformed error will be large due to lack of DC-coefficient correction scheme. In the typical design, as the DCT/IDCT are implemented with the same circuit, the cosine coefficient needs over 12 bits to meet the IDCT accuracy. For only DCT implementation, the cosine coefficients also need about 8 bits. From Table 3 and 4, when their cosine coefficients are less than 8 bits, the transformed error shall become serious if without using DC correction. Clearly our cost-effective design can save the word length while keeping the same coding quality.

Table 10 lists the comparisons with the other DCT processors. First we attend to attend the memory requirement about DCT IP. The line memory is required for block data access. The size of line RAM is about 46 k ($720 \times 8 \times 8$) bits for 720 samples each scanning line. For data transposition, the conventional row/column DCT processor

Table 9 The chip feature of the proposed DCT processor.

Items	Content
Process	TSMC 0.35UM 1P4M
Area	4 mm^2
No of transistors	19 k
Power dissipation	15 mW
Input pins (data + clock)	$38 (= 36 + 2)$
Output coefficient pins	$12 \times 2 = 24$
Power pins	6
Work voltage	3.3 V
Max. Operation Freq.	60 MHz

Table 10 Comparisons with other 2D-DCT processors for 8×8 block.

Transposed cell	[11] $2N^2 \times 16$ Embedded RAM	[12] $N^2 \times 16$ Register	[13] $(N^2 + N) \times 16$ Shift Register	[14] $2N^2 \times 16$ Register	[15] $N^2 \times 16$ Register	Proposed No
Memory address generator	Need	Need	No	Need	Need	No
Input buffer	$N^2/8 \times 9$	$N^2/8 \times 9$	$N^2/8 \times 9$	$N^2/8 \times 9$	$N^2/8 \times 9$	$N^2/4 \times 9$
Output register	No	$N^2/4 \times 16$	No	$N^2/8 \times 16$	$N^2/8 \times 16$	$N^2/8 \times 16$
Transistors in chip	102 K	67 K	—	101 K	—	19 K
(Estimated complexity)			(13,696)		(2320)	(671)
Complexity factor	5.3	3.5	20	5.3	3.5	1
Latency time (Cycles)	128	128	9	96	49	22
Throughput rate (#No Coeff./per cycle)	1	1	N	1	1	1

requires the transpose memory that used two sets of $N \times N$ cells for real-time operations [11, 14]. The first $N \times N$ memory cells are for writing the results of the 1st DCT. The second $N \times N$ cells are for reading the 1st DCT results to the 2nd DCT processing. The transposed memory requires $2N^2 \times \text{word-length}$ cells for real-time operation. The transposed cell can be realized with either embedded-RAM or register. With register-based implementation [12, 14, 15], the circuit design and simulation is easy, but we need large silicon area. Instead, one can employ embedded RAM rather than register to reduce the chip size. To reduce control overhead, the shift-register approach is used to realize the data transposition [13]. With this method, we do not require the address generator to access the memory content. For further reducing the transposed cell size, the [12] employs a common 1D-DCT unit to reduce a half of transposed memory. In the proposed DCT, the transposed memory and address generator can be saved. However, we need the input-buffers and the temporal output registers with $(N^2/4) \times 9$ and $(N^2/8) \times 16$ cells, respectively. The other DCT chips also require the input register for data ordering and the output register to accumulate the results. Clearly, the proposed DCT can reduce the memory cell.

For DCT implementation, most of architectures require ROM table to store the cosine coefficients [11–15]. From (7) and (8), we require two sets of 4×4 cosine matrix. If the cosine coefficients are presented with 8 bits, we require $8 \times 32 = 256$ memory cells. Besides, the address generator is needed to read the ROM table. In the proposed DCT chip, the state machine is used to generate the cosine coefficients instead of ROM table. The state machine uses n bits to produce 2^n values, such that $(2^n - n)$ memory cells can be saved. From Table 6 and 7, we only employ three registers to generate eight states. For H1~H4 and T1~T4 implementation, the total only requires 36 register-cells. The read control used a 3-bit counter rather than the address generator. Our circuit only employs about 19 K transistors

that are much smaller than the other DCT chips [11, 12, 14]. However, some DCT architectures [13, 15] do not show the number of transistors used in their lectures. To compare the performance, we can utilize the number of adders and registers to evaluate their complexity. The estimated complexity is defined by the accumulation of the number of 1-bit adder and 1-bit register since the adder and register dominate the size of a DCT chip. While a multiplier is used, it is equal to 16×1 -bit adders. For example, the circuit used two 16-bit multipliers, eight 16-bit adders and ten 8-bit registers, the result of estimated complexity is $2 \times 16 \times 16 + 8 \times 16 + 10 \times 8 = 720$. In our proposed chip, the 1st and 2nd DCT core needs 151 and 192 adder-bits, 124 and 204 register-bits, respectively. The estimated complexity of our chip is 671, but [13] and [15] requires 13,696 and 2,320 respectively. From results, the complexity factor of proposed architecture can be reduced at least 1/3 compared with the current DCT chips [11–15].

4 Conclusions

This paper proposed a cost-effective DCT processor for image/video coding applications. With parallel VLSI design, this DCT processor is capable to transform N coefficients during N cycles to meet real-time requirement. In summary, this DCT architecture has some advantages. (1) The particular computing schedule is proposed based on fast row/column algorithm, to save the transposed memory. (2) With DC-coefficient correction approach, the system word length can be shortened while keeping sufficient accuracy. As the cosine coefficient only uses 4-bit, the hardware expenses the lowest cost for practical JPEG/MPEG coding in experiments. (3) The state machine is used to generate the cosine coefficients, which not only can reduce the memory cells but also can save address generator. (4) The parallel structure with the multiplierless

architecture proposed only needs 36 adders for high-speed and low-cost implementation. The throughput of the chip can achieve 60 MHz in only use of 19 k transistors. Therefore, the high efficiency DCT processor would become a useful IP for the whole video encoding realization in SOC (system on one chip) design.

References

1. Taylor, G. S., & Blair, G. M. (1998). Design for the discrete cosine transform in VLSI. *IEEE Proceedings on Computer Digital Technology*, 145(2), 127–133. doi:10.1049/ip-cdt:19981911.
2. Wallace, G. K. (1991). The JPEG still picture compression standard. *Communications of the ACM*, 34(4), 30–44. doi:10.1145/103085.103089.
3. MPEG-2 video coder, ISO/IEC DIS 13818-2.
4. Cote, G., Erol, B., & Kossentini, F. (1998). H.263+: video coding at low bit rate. *IEEE Transactions on Circuits and Systems for Video Technology*, 8(7), 849–866. doi:10.1109/76.735381.
5. Feig, E., & Winograd, S. (1992). Fast algorithm for the discrete cosine transform. *IEEE Transactions on Signal Processing*, 40 (Sept), 2174–2193. doi:10.1109/78.157218.
6. Li, W. (1991). A new algorithm to compute the DCT and its inverse. *IEEE Transactions on Signal Processing*, 39(6), 1035–1313. doi:10.1109/78.136537.
7. Cho, N. I., & Lee, S. U. (1991). Fast algorithm and implementation of 2-D discrete cosine transform. *IEEE Transactions on Circuits and Systems*, 38(3), 297–305. doi:10.1109/31.101322.
8. Slaweck, D., & Li, W. (1992). DCT/IDCT processor design for high data rate coding. *IEEE Transactions on Circuits and Systems for Video Technology*, 2(2), 135–145. doi:10.1109/76.143413.
9. Hsia, S. C., Liu, B. D., Yang, J. F., & Bai, B. L. (1992). VLSI implementation of parallel coefficient-by-coefficient two-dimensional IDCT processor. *IEEE Transactions on Circuits and Systems for Video Technology*, 5(5), 396–406 Oct. 1995.
10. Chang, Y. T., & Wang, C. L. (1992). New systolic array implementation of the 2D discrete cosine transform and its inverse. *IEEE Transactions on Circuits and Systems for Video Technology*, 5(2), 150–157.
11. Uramoto, S. I., Inoue, Y., Takabake, A., Yamashita, Y., Terane, H., & Yoshimoto, M. (1992). A 100 MHz 2-D discrete cosine transform core processor. *IEEE Journal of Solid-State Circuits*, 27(4), 492–498. doi:10.1109/4.126536.
12. Madisetti, A., & Willson, N. (1995). A 100 MHz 2-D 8×8 DCT/IDCT processor for HDTV applications. *IEEE Transactions on Circuits and Systems for Video Technology*, 5(2), 158–164. doi:10.1109/76.388064.
13. Aggoun, A., & Jalloh, I. (2003). Two-dimensional DCT/IDCT architecture. *IEE Proceedings-Computers and Digital Techniques*, 150(1), 2–10 20 Jan.
14. Kyeounsoo, K. i. m., & Jong-Seog, K. o. h. (1999). An area efficient DCT architecture for MPEG-2 video encoder. *IEEE Transactions on Consumer Electronics*, 45(1), 62–67 Feb.
15. Takala, J., Nikara, J., Akopian, D., Astola, J., & Saarinen, J. (2000). Pipeline architecture for 8*8 discrete cosine transform. *IEEE International Conference on Acoustics, Speech, and Signal Processing, 2000. ICASSP '00. Proc.*, 6, 3303–3306.
16. Hsia S. C. (2003). An adaptive video coding control scheme for real-time MPEG applications. *Journal on Applied Signal Processing*, 245–251, Mar.
17. Hsia, S.-C. (2003). Efficient memory IP design for HDTV coding applications. *IEEE Transactions on Circuits and Systems for Video Technology*, 13(6), 465–471. doi:10.1109/TCSVT.2003.813418.
18. Samir, P. (1996). “Verilog HDL,” Prentice Hall.



Shih-Chang Hsia received the Ph.D. degrees from the Department of Electrical Engineering, National Cheng Kung University, Tainan, Taiwan, ROC, in 1996. During 1986–1989, he was an engineer in the R&D department of Microtek International Inc., Hsin-Chu. He was an instructor and associate professor in the Department of Electronic Engineering, Chung Chou Institute of Technology during 1991–1997. Since 1997, he joined Department of Computer and Communication Engineering, National Kaohsiung First University of Science and Technology Kaohsiung. Currently, he is a professor in Department of Computer and Communication Engineering, and Department of Electronics Engineering, at the same school. He was elected as a chairman at Department of Electronics Engineering since 2007. His research interests include VLSI/SOC designs, video/image coding and processing, HDTV/DTV systems, communication system and data hiding system.



Chin-Feng Tsai received the M.S. degree in Computer and Communication Engineering from National Kaohsiung First University of Science and Technology, Taiwan, R.O.C., in 2000. He is a Ph.D.

candidate in National Kaohsiung First University of Science and Technology, Taiwan. His research interests include wavelet transformation and neural network.



Szu-Hong Wang received the M.S. degree from the Department of Computer and Communication Engineering, National Kaohsiung First University of Science and Technology, Kaohsiung, Taiwan, in 2005. He is currently pursuing the Ph.D degree at the National Kaohsiung First University of Science and Technology, Kaohsiung, Taiwan. His major research interests include H.264/AVC video coding and VLSI design.



King-Chu Hung was born in Tainan, Taiwan, in 1959. He received the B.S., M.S., and Ph.D. degrees in electrical engineering from National Cheng Kung University, Tainan, in 1980, 1982, and 1988, respectively. In 1988, he was an associate professor with the Institute of Computer Science and Electronic Engineering, National Central University, Taiwan. From 1989 to 1995, he was an associate researcher and technology supervisor with Chung Shan Institute of Science and Technology (CSIST), Lung Tan, Taiwan. Since 1995, he was an associate professor of Electronic Engineering with I-Shou University, Taiwan. Currently he is a professor of the Department of Computer and Communication Engineering at the National Kaohsiung First University of Science and Technology, Taiwan. His current research interests include computer vision, mathematics, image compression, VLSI, wavelets, and error control coding.